

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И
ИНФОРМАТИКИ

Кафедра «Математическая кибернетика и информационные технологии»

Лабораторная работа № 3
по дисциплине «Введение в информационные технологии»
по теме: «Функции в Python и базовые алгоритмы»

Выполнил студент группы:

БВТ2604

Карас Михаил Сергеевич

Проверил: Павликов Артем Евгеньевич

Москва 2026

Оглавление

Цель и задачи работы:.....	3
Выполнение	3
Вывод.....	16

Цель и задачи работы:

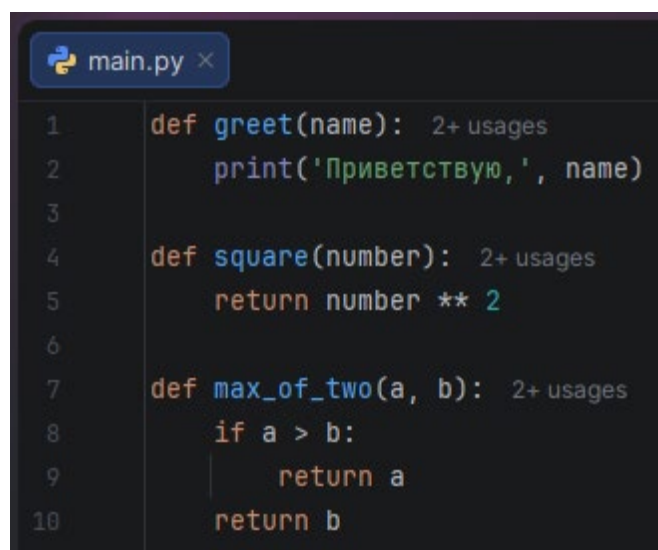
Изучить принципы создания и использования функций в языке Python, научиться передавать данные в функции и возвращать результаты, понять области видимости переменных, освоить работу со списками и кортежами, срезами и встроенными функциями, а также получить практический опыт реализации базовых алгоритмов обработки последовательностей и применения lambda-функций.

В ходе выполнения лабораторной работы необходимо научиться разбивать программу на логически завершённые функции, использовать параметры и возвращаемые значения, обрабатывать последовательности данных и самостоятельно реализовывать простейшие алгоритмы поиска, подсчёта и агрегирования.

Выполнение

1. Создание и вызов функций

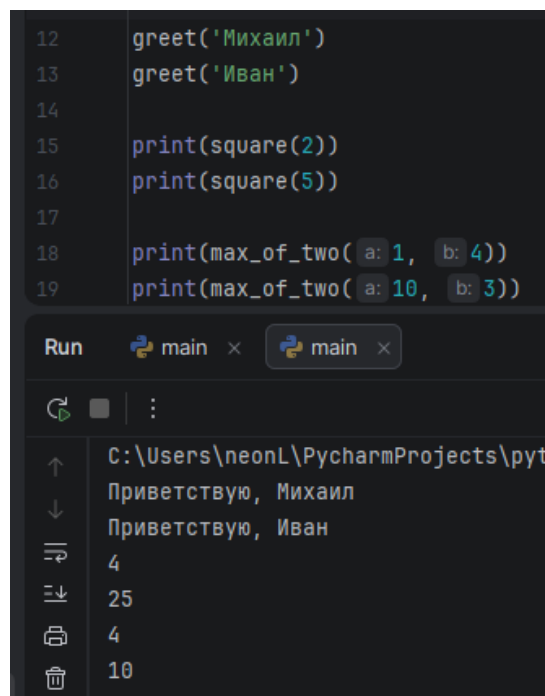
Создал функции `greet(name)`, `square(number)` и `max_of_two(a, b)`. Функция `greet()` выводит приветствие, `square()` возвращает квадрат переданного числа, а `max_of_two()` возвращает большее из двух значений без использования встроенной функции `max()`.



```
main.py x
1 def greet(name): 2+ usages
2     print('Приветствую,', name)
3
4 def square(number): 2+ usages
5     return number ** 2
6
7 def max_of_two(a, b): 2+ usages
8     if a > b:
9         return a
10    return b
```

Фото №1. Код функций `greet`, `square` и `max_of_two`.

Далее вызвал каждую функцию не менее двух раз с различными аргументами.



The screenshot shows a Python IDE with a dark theme. The editor window displays the following code:

```
12 greet('Михаил')
13 greet('Иван')
14
15 print(square(2))
16 print(square(5))
17
18 print(max_of_two(a: 1, b: 4))
19 print(max_of_two(a: 10, b: 3))
```

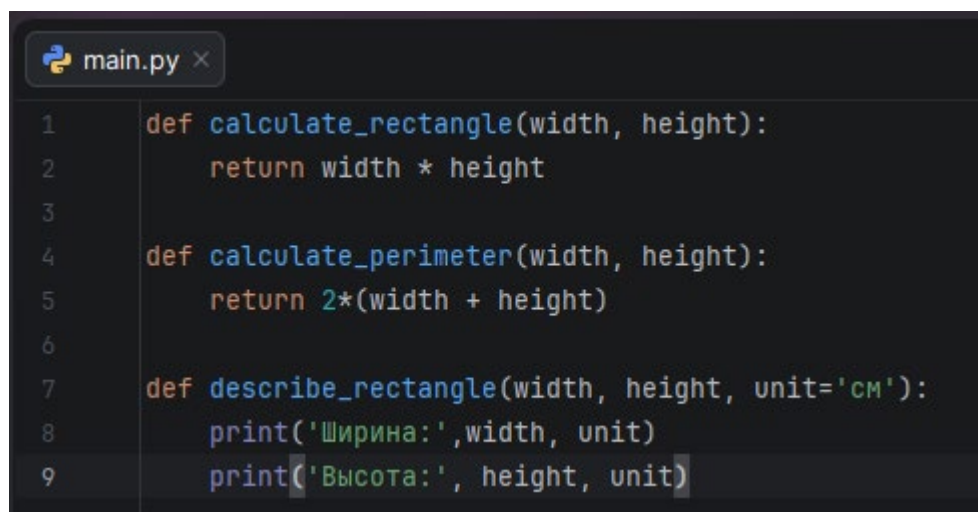
Below the editor, the 'Run' button is visible. The output console shows the results of the execution:

```
Приветствую, Михаил
Приветствую, Иван
4
25
4
10
```

Фото №2. Результаты вызова функций.

2. Аргументы и возвращаемые значения

Создал функции `calculate_rectangle(width, height)` и `calculate_perimeter(width, height)`, возвращающие площадь и периметр прямоугольника, а также функцию `describe_rectangle(width, height, unit="см")`, использующую единицу измерения по умолчанию.

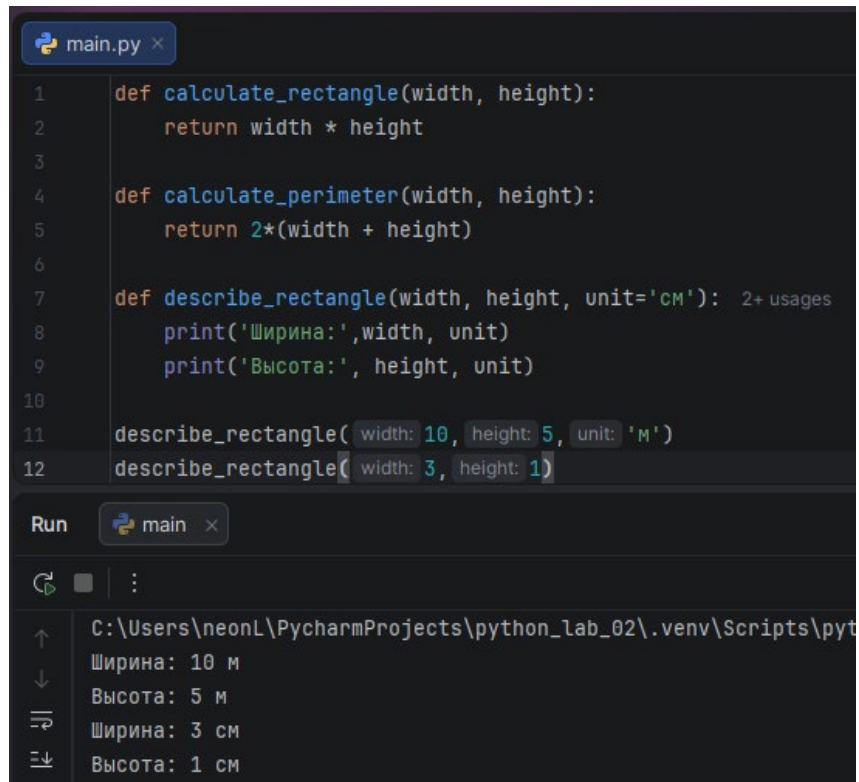


The screenshot shows a Python IDE with a dark theme. The editor window displays the following code:

```
1 def calculate_rectangle(width, height):
2     return width * height
3
4 def calculate_perimeter(width, height):
5     return 2*(width + height)
6
7 def describe_rectangle(width, height, unit='см'):
8     print('Ширина:', width, unit)
9     print('Высота:', height, unit)
```

Фото №3. Код функций для работы с прямоугольником.

Продемонстрировал вызов функции `describe_rectangle()` как с аргументом `unit`, так и без него.



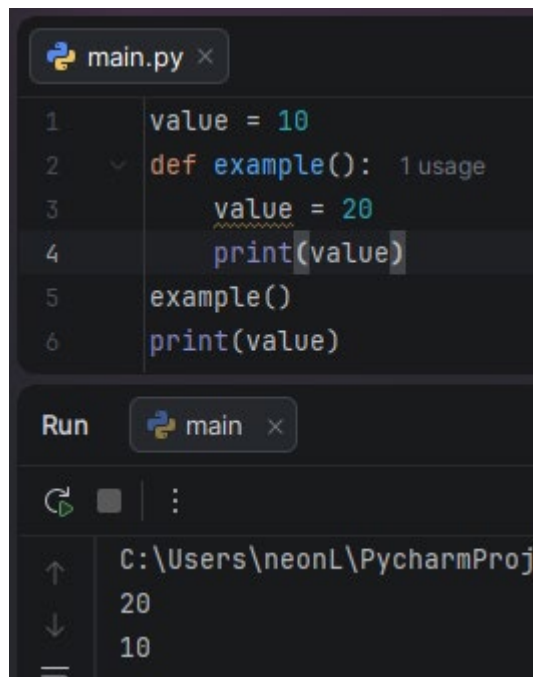
```
main.py x
1 def calculate_rectangle(width, height):
2     return width * height
3
4 def calculate_perimeter(width, height):
5     return 2*(width + height)
6
7 def describe_rectangle(width, height, unit='см'): 2+ usages
8     print('Ширина:', width, unit)
9     print('Высота:', height, unit)
10
11 describe_rectangle(width: 10, height: 5, unit: 'м')
12 describe_rectangle(width: 3, height: 1)

Run main x
C:\Users\neonL\PycharmProjects\python_lab_02\.venv\Scripts\pyt
Ширина: 10 м
Высота: 5 м
Ширина: 3 см
Высота: 1 см
```

Фото №4. Вызов `describe_rectangle()` с аргументом `unit` и без него.

3. Области видимости переменных

Выполнил программу, в которой переменная `value` определена в глобальной области видимости и внутри функции `example()`.



```
main.py x
1 value = 10
2 def example(): 1 usage
3     value = 20
4     print(value)
5 example()
6 print(value)

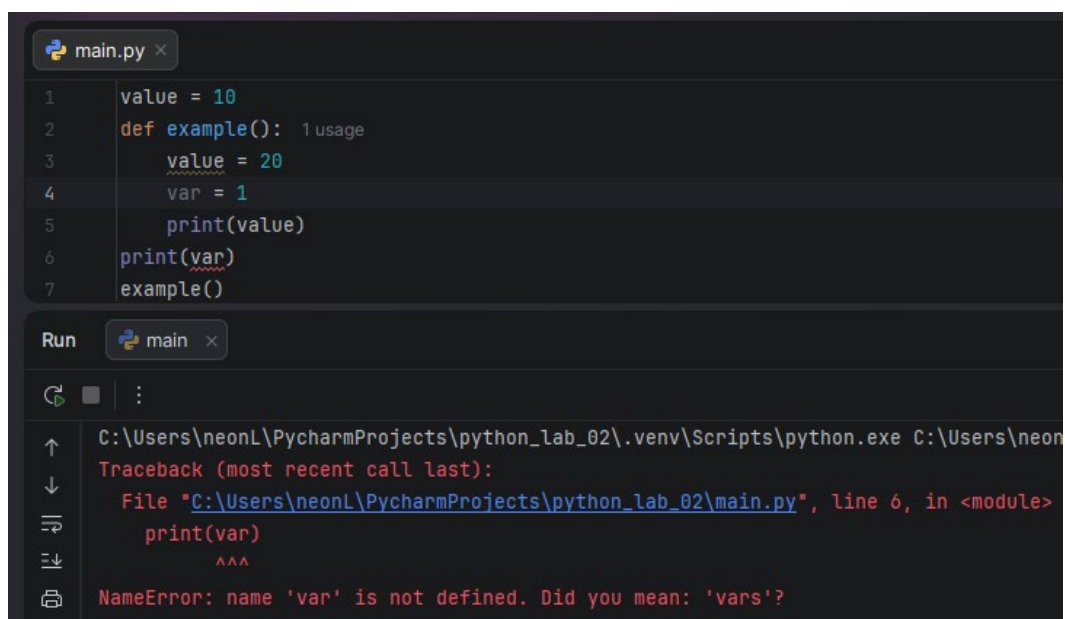
Run main x

C:\Users\neonL\PycharmProj
20
10
```

Фото №5. Результат выполнения программы.

Два разных значения выводятся из-за того, что переменная `value`, которая находится внутри функции – локальная и действует только пока выполняется сама функция, не изменяя глобальные переменные с таким же именем вне функции.

Далее добавил внутри функции ещё одну локальную переменную и попытался обратиться к ней после вызова функции.



```
main.py x
1 value = 10
2 def example(): 1 usage
3     value = 20
4     var = 1
5     print(value)
6     print(var)
7 example()

Run main x

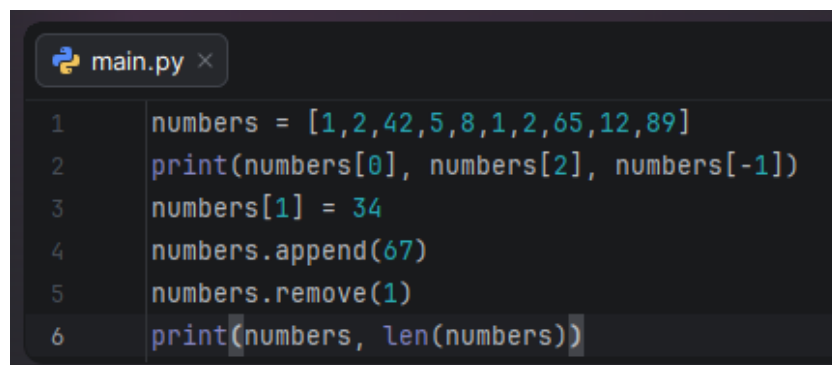
C:\Users\neonL\PycharmProjects\python_lab_02\.venv\Scripts\python.exe C:\Users\neonL\PycharmProjects\python_lab_02\main.py
Traceback (most recent call last):
  File "C:\Users\neonL\PycharmProjects\python_lab_02\main.py", line 6, in <module>
    print(var)
    ^^^^^
NameError: name 'var' is not defined. Did you mean: 'vars'?
```

Фото №6. Попытка обращения к локальной переменной за пределами функции.

Программа выдала ошибку, так как к локальной переменной `var`, созданной внутри функции, невозможно обратиться за пределами этой функции. Для работы с глобальной переменной внутри функции используется ключевое слово `global`.

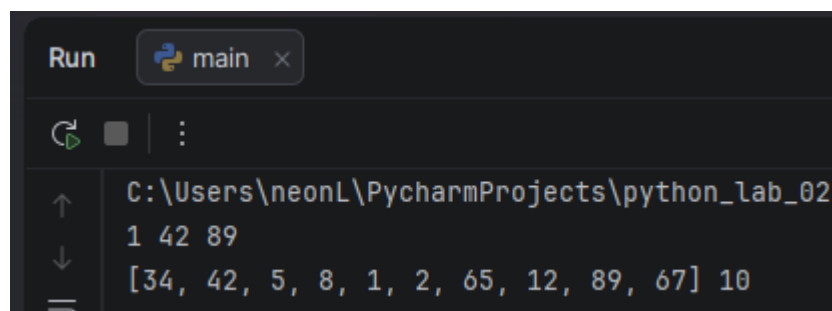
4. Работа со списками

Создал список, содержащий не менее 10 целых чисел. Вывел первый, третий и последний элементы, изменил значение второго элемента, добавил новый элемент с помощью `append()` и удалил один из элементов с помощью `remove()` или `pop()`. После этого вывел итоговый список и количество его элементов.



```
main.py x
1 numbers = [1,2,42,5,8,1,2,65,12,89]
2 print(numbers[0], numbers[2], numbers[-1])
3 numbers[1] = 34
4 numbers.append(67)
5 numbers.remove(1)
6 print(numbers, len(numbers))
```

Фото №7. Программа для работы со списком.

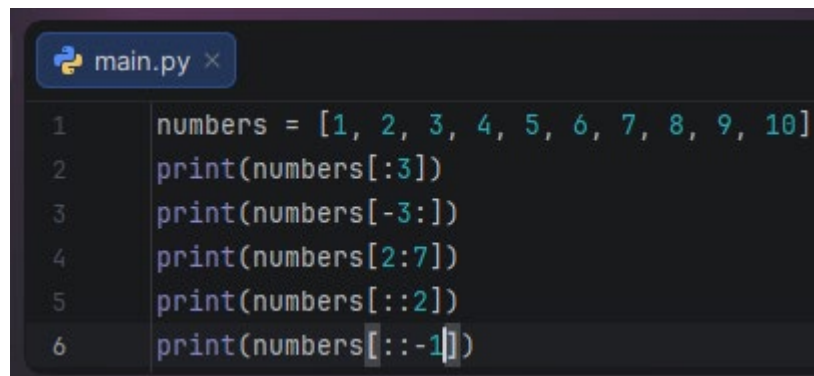


```
Run main x
C:\Users\neonL\PycharmProjects\python_lab_02\
1 42 89
[34, 42, 5, 8, 1, 2, 65, 12, 89, 67] 10
```

Фото №8. Результат работы программы со списком.

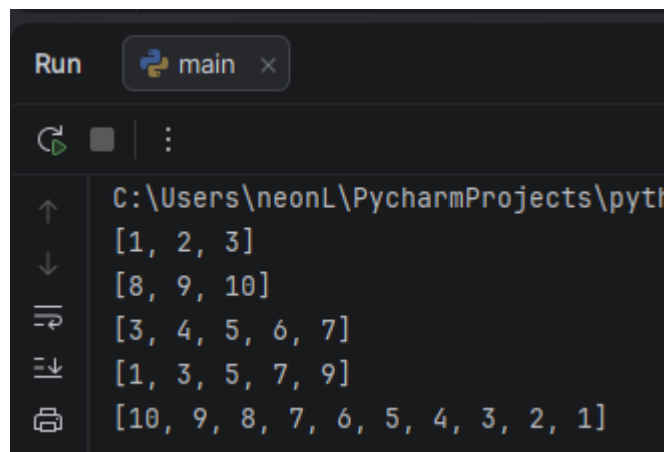
5. Срезы

Для списка `numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]` с помощью срезов получил и вывел первые три элемента, последние три элемента, элементы с 3-го по 7-й, каждый второй элемент и список в обратном порядке.



```
main.py x
1 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2 print(numbers[:3])
3 print(numbers[-3:])
4 print(numbers[2:7])
5 print(numbers[::2])
6 print(numbers[::-1])
```

Фото №9. Программа со срезами списка.

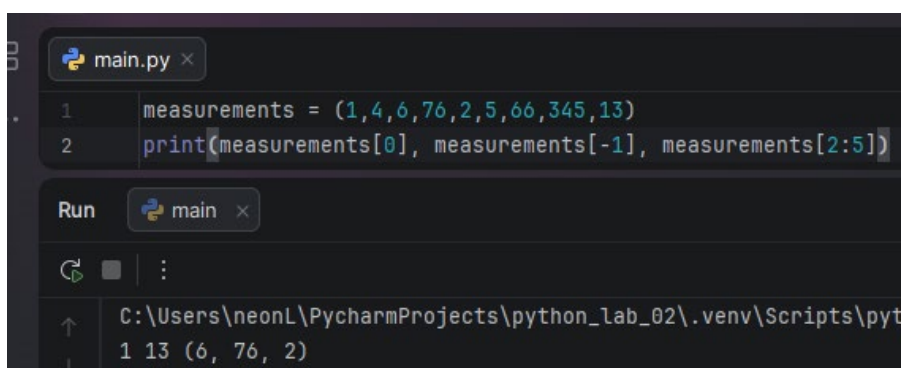


```
Run main x
C:\Users\neonL\PycharmProjects\pyth
[1, 2, 3]
[8, 9, 10]
[3, 4, 5, 6, 7]
[1, 3, 5, 7, 9]
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Фото №10. Результат выполнения срезов.

6. Работа с кортежами

Создал кортеж `measurements`, содержащий не менее 8 числовых значений, и вывел первый и последний элементы, а также несколько элементов с помощью среза.

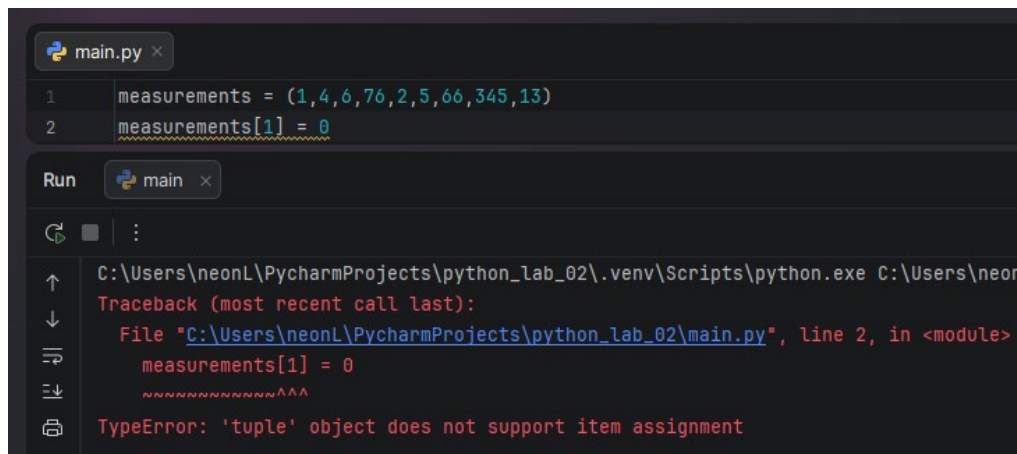


```
main.py x
1 measurements = (1,4,6,76,2,5,66,345,13)
2 print(measurements[0], measurements[-1], measurements[2:5])

Run main x
C:\Users\neonL\PycharmProjects\python_lab_02\.venv\Scripts\pyt
1 13 (6, 76, 2)
```

Фото №11. Создание кортежа и вывод его элементов.

Далее попытался изменить один из элементов кортежа.



```
main.py x
1 measurements = (1,4,6,76,2,5,66,345,13)
2 measurements[1] = 0

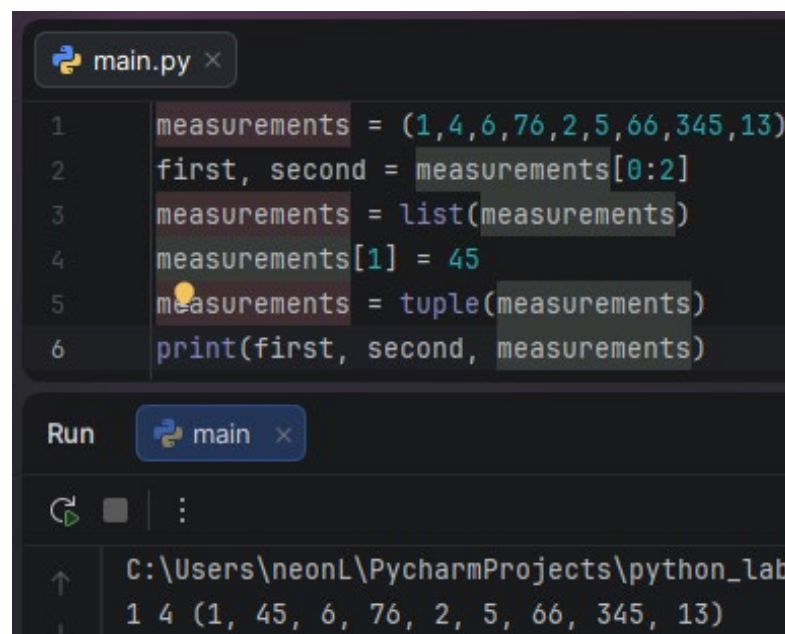
Run main x

C:\Users\neonL\PycharmProjects\python_lab_02\.venv\Scripts\python.exe C:\Users\neonL\PycharmProjects\python_lab_02\main.py
Traceback (most recent call last):
  File "C:\Users\neonL\PycharmProjects\python_lab_02\main.py", line 2, in <module>
    measurements[1] = 0
    ~~~~~^~~~~~
TypeError: 'tuple' object does not support item assignment
```

Фото №12. Ошибка при попытке изменить элемент кортежа.

Ошибка возникает из-за того, что кортеж — это неизменяемая последовательность, элементы в которой нельзя редактировать. Если необходимо изменить элементы кортежа, его можно преобразовать в список, внести изменения и затем снова преобразовать в кортеж.

После этого распаковал первые два значения кортежа в отдельные переменные, преобразовал кортеж в список, изменил один элемент и преобразовал результат обратно в кортеж.



```
main.py x
1 measurements = (1,4,6,76,2,5,66,345,13)
2 first, second = measurements[0:2]
3 measurements = list(measurements)
4 measurements[1] = 45
5 measurements = tuple(measurements)
6 print(first, second, measurements)

Run main x

C:\Users\neonL\PycharmProjects\python_lab_02\main.py
1 4 (1, 45, 6, 76, 2, 5, 66, 345, 13)
```

Фото №13. Распаковка кортежа и его преобразование в список и обратно.

7. Встроенные функции для работы с последовательностями

Для созданного ранее списка чисел с помощью встроенных функций определил количество элементов, сумму элементов, минимальное и максимальное значения, а также среднее арифметическое.



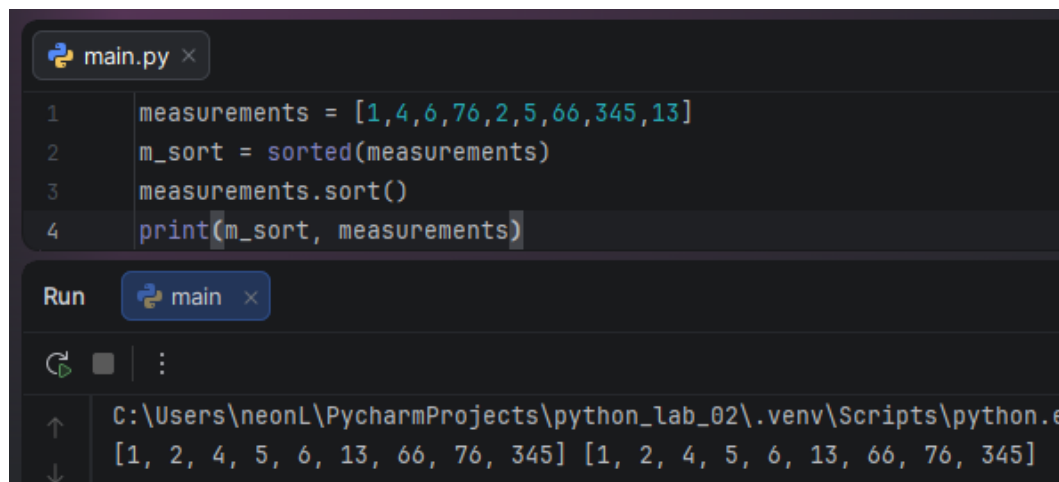
```
main.py x
1 measurements = [1,4,6,76,2,5,66,345,13]
2 avg = sum(measurements)/len(measurements)
3 print('Количество:', len(measurements), '\nСумма:', sum(measurements), '\nМинимальное:',
4       sum(measurements), '\nМаксимальное:', sum(measurements), '\nСр.арифм.:', avg)

Run main x

C:\Users\neonL\PycharmProjects\python_lab_02\.venv\Scripts\python.exe C:\Users\neonL\Pychar
Количество: 9
Сумма: 518
Минимальное: 518
Максимальное: 518
Ср.арифм.: 57.555555555555556
```

Фото №14. Характеристики списка, вычисленные встроенными функциями.

Далее получил отсортированную копию списка с помощью `sorted()`, после чего отсортировал сам список методом `sort()`.



```
main.py x
1 measurements = [1,4,6,76,2,5,66,345,13]
2 m_sort = sorted(measurements)
3 measurements.sort()
4 print(m_sort, measurements)

Run main x

C:\Users\neonL\PycharmProjects\python_lab_02\.venv\Scripts\python.exe
[1, 2, 4, 5, 6, 13, 66, 76, 345] [1, 2, 4, 5, 6, 13, 66, 76, 345]
```

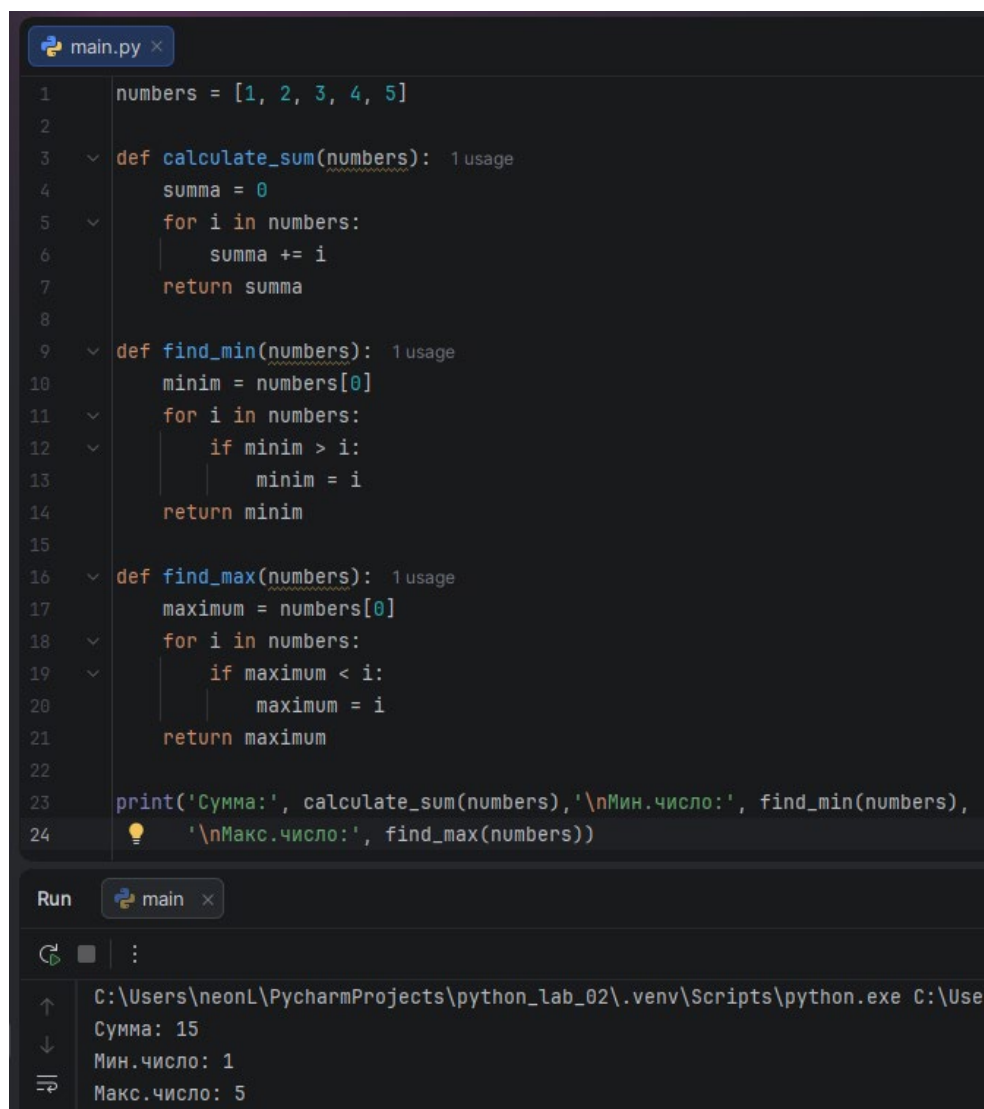
Фото №15. Результаты применения `sorted()` и `sort()`.

Разница между этими результатами в том, что метод `sorted()` создает новый отсортированный список, а `sort()` изменяет уже существующий.

8. Реализация собственных алгоритмов

Без использования встроенных функций `sum()`, `min()` и `max()` реализовал функции `calculate_sum(numbers)`, `find_min(numbers)` и `find_max(numbers)`.

Каждая функция принимает список чисел и возвращает соответствующий результат.



```
1 numbers = [1, 2, 3, 4, 5]
2
3 def calculate_sum(numbers): 1 usage
4     summa = 0
5     for i in numbers:
6         summa += i
7     return summa
8
9 def find_min(numbers): 1 usage
10    minim = numbers[0]
11    for i in numbers:
12        if minim > i:
13            minim = i
14    return minim
15
16 def find_max(numbers): 1 usage
17    maximum = numbers[0]
18    for i in numbers:
19        if maximum < i:
20            maximum = i
21    return maximum
22
23 print('Сумма:', calculate_sum(numbers), '\nМин.число:', find_min(numbers),
24       '\nМакс.число:', find_max(numbers))
```

Run main

C:\Users\neonL\PycharmProjects\python_lab_02\.venv\Scripts\python.exe C:\Use
Сумма: 15
Мин.число: 1
Макс.число: 5

Фото №16. Код функций calculate_sum, find_min и find_max.

Сравнил полученные значения с результатами встроенных функций Python.

```
23     print('Сумма:', calculate_sum(numbers), sum(numbers),
24           '\nМин.число:', find_min(numbers), min(numbers),
25           '\nМакс.число:', find_max(numbers), max(numbers))
```

Run main x

С:\Users\neonL\PycharmProjects\python_lab_02\.venv\Scripts
Сумма: 15 15
Мин.число: 1 1
Макс.число: 5 5

Фото №17. Сравнение результатов собственных и встроенных функций.

9. Линейный поиск

Реализовал функцию `find_element(numbers, target)`, которая последовательно просматривает список и возвращает индекс первого найденного элемента. Если значение отсутствует, функция возвращает -1.

```
main.py x
1     numbers = [1, 2, 3, 4, 5]
2
3     def find_element(numbers, target): 1 usage
4         for i in range(len(numbers)):
5             if numbers[i] == target:
6                 return i
7         return -1
```

Фото №18. Код функции `find_element`.

Работа алгоритма была проверена для существующего и отсутствующего значения.

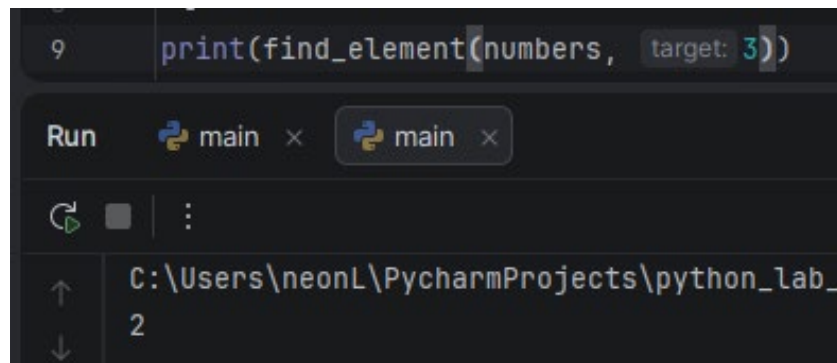


Фото №19. Поиск существующего значения.

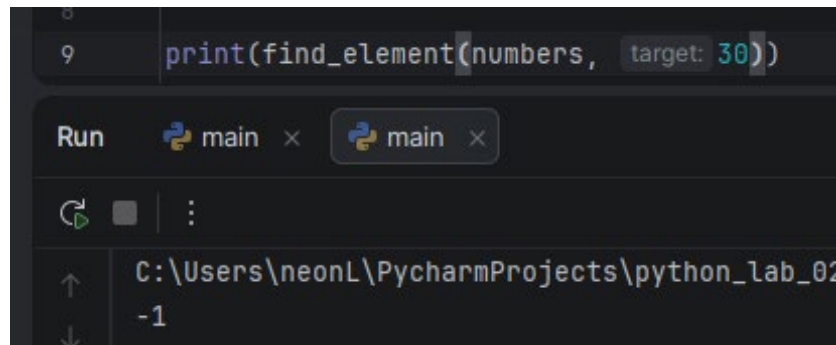


Фото №20. Поиск отсутствующего значения.

10. Возврат нескольких значений

Создал функцию `analyze(numbers)`, которая принимает список чисел и возвращает кортеж из минимального значения, максимального значения и среднего арифметического. Вызвал функцию, распаковал возвращённый кортеж в три отдельные переменные и вывел результаты на экран.

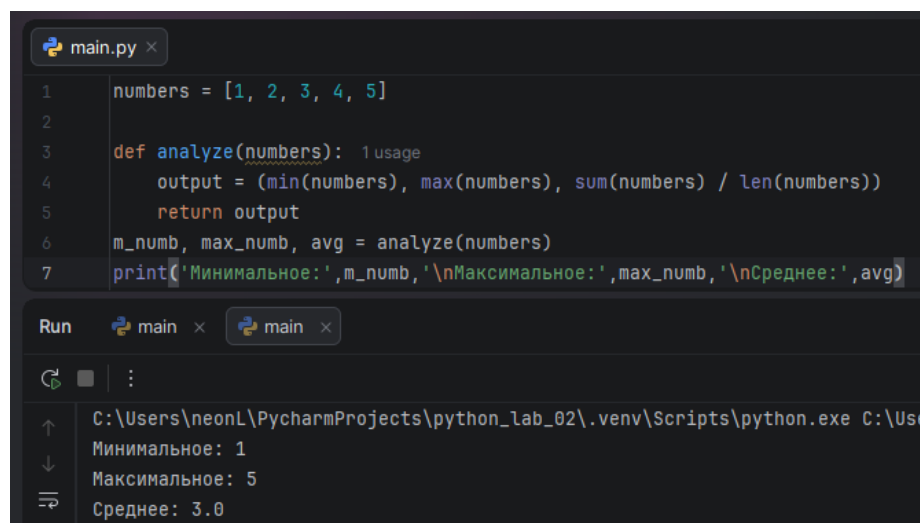
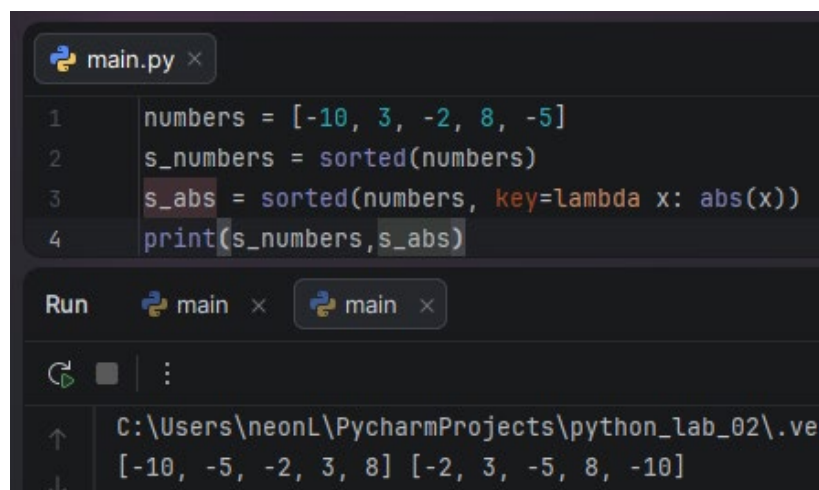


Фото №21. Программа с функцией `analyze` и результат её выполнения.

11. Lambda-функции

Для списка `numbers = [-10, 3, -2, 8, -5]` получил обычную сортировку с помощью `sorted()`, а затем сортировку по абсолютному значению элементов с использованием параметра `key` и `lambda`-функции.



```
main.py x
1 numbers = [-10, 3, -2, 8, -5]
2 s_numbers = sorted(numbers)
3 s_abs = sorted(numbers, key=lambda x: abs(x))
4 print(s_numbers, s_abs)

Run main x main x

C:\Users\neonL\PycharmProjects\python_lab_02\ven
[-10, -5, -2, 3, 8] [-2, 3, -5, 8, -10]
```

Фото №22. Сортировка списка обычным способом и с использованием `lambda`-функции.

Результаты отличаются тем, что вторая функция берёт модуль от значения элемента, то есть сортирует числа по их абсолютному значению.

12. Самостоятельная работа

В ходе самостоятельной работы была написана программа для анализа набора результатов измерений. Исходные данные хранятся в списке, содержащем не менее 10 числовых значений. Программа содержит отдельные функции `calculate_average()`, `find_minimum()`, `find_maximum()`, `count_above_average()`, `find_value()` и `analyze_measurements()`. В функциях `calculate_average()`, `find_minimum()` и `find_maximum()` не используются встроенные функции `sum()`, `min()` и `max()`.

Функция `analyze_measurements()` возвращает кортеж, содержащий минимум, максимум и среднее значение, который затем распаковывается в отдельные переменные. Программа выводит количество измерений, минимальное, максимальное и среднее значения, количество значений выше среднего и результат поиска значения, заданного пользователем.

```

main.py x
1  measurements = [1, 34, 53, 564, 12, 34, 5, 4, 25, 76]
2
3  def calculate_average(measurements):  2+ usages
4      summa = 0
5      n = 0
6      for i in measurements:
7          summa += i
8          n += 1
9      return summa/n
10
11 def find_minimum(measurements):  1 usage
12     minim = measurements[0]
13     for i in measurements:
14         if i < minim:
15             minim = i
16     return minim
17
18 def find_maximum(measurements):  1 usage
19     maximum = measurements[0]
20     for i in measurements:
21         if i > maximum:
22             maximum = i
23     return maximum
24
25 def count_above_average(measurements):  1 usage
26     k = 0
27     avg = calculate_average(measurements)
28     for i in measurements:
29         if i > avg:
30             k+=1
31     return k
32
33 def find_value(measurements, target):  1 usage
34     for i in range(len(measurements)):
35         if measurements[i] == target:
36             return i
37     return -1

def analyze_measurements(measurements):  1 usage
    output = (find_minimum(measurements),
              find_maximum(measurements),
              calculate_average(measurements))
    return output

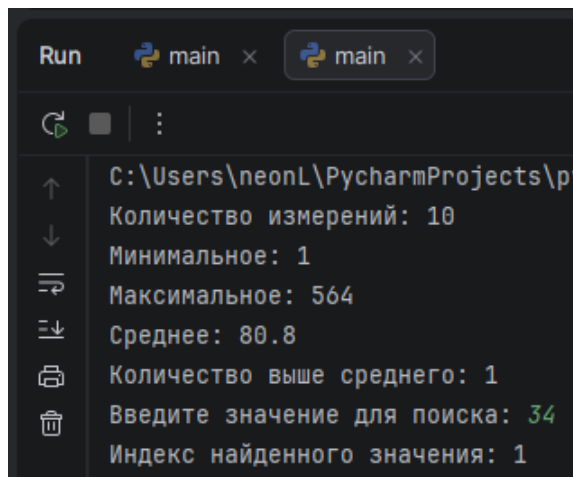
minimum, maximum, average = analyze_measurements(measurements)

print('Количество измерений:', len(measurements))
print('Минимальное:', minimum)
print('Максимальное:', maximum)
print('Среднее:', average)
print('Количество выше среднего:', count_above_average(measurements))

target = int(input('Введите значение для поиска: '))
print('Индекс найденного значения:', find_value(measurements, target))

```

Фото №23, 24. Код программы для анализа результатов измерений.

A screenshot of a PyCharm Run console window. The window has a dark theme. At the top, there are tabs labeled 'Run' and 'main'. Below the tabs, there is a green play button icon and a square icon. The main area of the console displays the following text: 'C:\Users\neonL\PycharmProjects\py', 'Количество измерений: 10', 'Минимальное: 1', 'Максимальное: 564', 'Среднее: 80.8', 'Количество выше среднего: 1', 'Введите значение для поиска: 34', and 'Индекс найденного значения: 1'. On the left side of the console, there are several icons for file operations: an upward arrow, a downward arrow, a double upward arrow, a double downward arrow, a copy icon, and a delete icon.

```
Run  main ×  main ×  
C:\Users\neonL\PycharmProjects\py  
Количество измерений: 10  
Минимальное: 1  
Максимальное: 564  
Среднее: 80.8  
Количество выше среднего: 1  
Введите значение для поиска: 34  
Индекс найденного значения: 1
```

Фото №25. Результат работы программы.

Вывод

В ходе лабораторной работы были изучены функции Python и базовые алгоритмы. Были реализованы функции для вычисления среднего значения, поиска минимума и максимума, подсчёта значений выше среднего и линейного поиска элемента. Также была создана функция, возвращающая несколько результатов в виде кортежа, и выполнена распаковка полученных значений. В результате были закреплены навыки работы с функциями, списками, циклами, условиями и кортежами.